YouTube Trending Videos Data Pipeline

Name: Soni Kumari
Course: Data Engineering / Data Pipelines
Main Reference: Google Cloud data pipeline article (by Patrick Alexander)

1. Selected Source

Source: YouTube Data API (public, free with an API key).

What type of data it gives:

- Video id, title and channel name
- Views, likes and comment count
- Category and published date
- Time when I collected the data

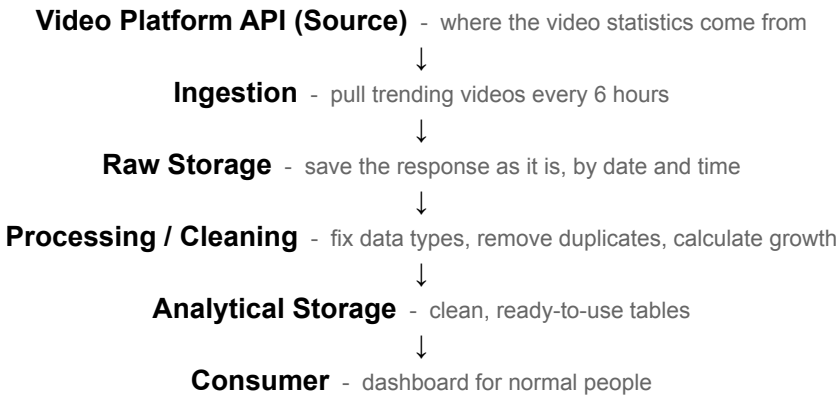
This is batch data. Views and likes keep growing, but I do not need every second. I will collect the list of trending videos in India every 6 hours. Example JSON looks like:

```
{
  "video_id":"abc123",
   "channel":"XYZ",
   "views":"1520000",
   "likes":"84000",
   "collected_at":"2026-10-05 10:00"
 }
```

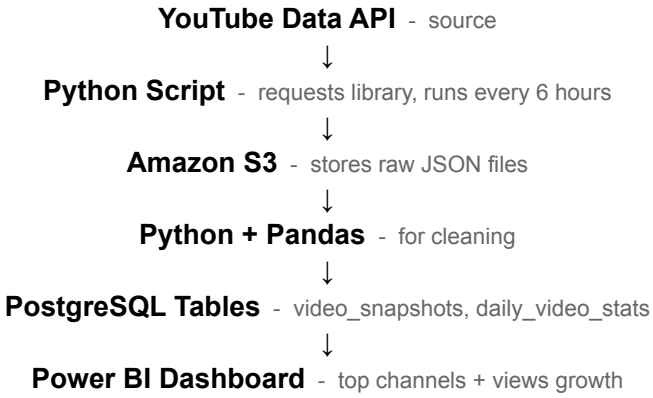
One thing I noticed: the API gives the counts as text (like '1520000') and not as numbers, so they need to be converted later.

Why this source? It is public, real, and easy to explain. It only has public video statistics, so there is no privacy problem. And it is different from a simple weather feed, because the numbers keep growing and I can compare snapshots.

2. Logical Architecture



3. Technology Mapping



This is my mapping:

- •Logical Ingestion = Python Script
- •Logical Raw Storage = S3
- •Logical Processing = Pandas
- •Logical Analytical Storage = PostgreSQL
- •Logical Consumer = Power BI

4. Component Explanation

Component

Component	Purpose
YouTube Data API	This is where the data is born. It gives public statistics of videos.
Python Script	It calls the API every 6 hours and brings data into my pipeline. YouTube will not send data by itself, and the API has a daily limit, so a dashboard cannot call it again and again.
Amazon S3	Used to store the raw JSON as a backup, so if my cleaning code has a bug I still have the original
Python + Pandas	Used to flatten the JSON, change text counts to numbers, remove duplicate rows, and calculate growth and engagement (likes + comments /
PostgreSQL	Used to store only the clean tables, so the dashboard can read fast with SQL.
Power BI Dashboard	**Normal users see charts here. They don't need to understand JSON or SQL.**

5. Question Log

1. Why can't Power BI directly call the YouTube API? Why do we need S3 and a database in between?
2. The API only shows the current views. How will I get growth over time?
3. Why save raw data? Can't we just clean and store the final data directly?
4. How often should I pull data? Every minute, every hour, or every day? What is the API limit?
5. What is the difference between ETL and ELT? Which one is my pipeline?
6. Do I really need Spark/Kafka here? My data is small, is Pandas not enough?

Conclusion:

The system relies on a six-hour snapshot approach to overcome the limitations of the direct API, which is slow, has a daily limit, and does not retain history. Raw data is stored first for safety, and then cleaned using Pandas, resembling an ELT workflow. This approach avoids the need for heavy tools like Spark or Kafka since the dataset is small, and six-hour intervals are sufficient for tracking trends within API limits.

6. Short Architecture Explanation

Why does data move from one box to the next?

Data starts at the YouTube API because that is the only place these statistics exist. Then the Python script pulls it every 6 hours and puts it in S3, because we need to bring outside data into our own control and keep every snapshot. Then data moves from S3 to Pandas for cleaning, because raw JSON is messy: counts are text, fields can be missing, and the same video appears in many snapshots. After cleaning, data moves to PostgreSQL, because the dashboard needs fast, clean rows, not big JSON files. At last, Power BI reads from PostgreSQL and shows charts, because normal people cannot read JSON or SQL, they need visuals.

API creates → Python brings → S3 saves safely → Pandas cleans → PostgreSQL keeps ready → Power BI shows.